

软件名称: AOI-Vision (AOI-Vision) 基于OasysCoreCV的自动光学检测系统软件
版本: V0.2.0
著作权人: 赵达伟 / 潤芯國際(香港)有限公司 / 昆山山之旭电子科技有限公司
源程序前30页 (共30页)

```
=====
// ===== 文件: aoi_engine/__init__.py =====
1 | """AOI Engine - 视觉核心引擎 (独立包)
2 | =====
3 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
4 | 描述: 所有 OasysCoreCV 算法在此包内, 与业务层/UI层零耦合。
5 |       可独立于 FastAPI 运行。SysAdmin 可通过 ModuleRegistry 按客户需求打包。
6 |       发布时用 Cython 编译为 .so, 保护核心算法。
7 |
8 | 商业化三层架构:
9 |   UI Layer (index.html) ←HTTP→ Business Layer (FastAPI) ←import→ Core
Engine (aoi_engine)
10 | """
11 | from aoi_engine.registry import ModuleRegistry, EngineModule, registry
12 | from aoi_engine.defect import DefectEngine, InspectionResult, Defect
13 | from aoi_engine.vision import VisionAnalyzer
14 | from aoi_engine.camera import BaseCamera, CameraProperties
15 | from aoi_engine.calibration import CalibrationEngine
16 |
17 | __all__ = [
18 |     "DefectEngine", "VisionAnalyzer", "BaseCamera", "CameraProperties",
"CalibrationEngine",
19 |     "ModuleRegistry", "EngineModule", "registry",
20 |     "InspectionResult", "Defect",
21 | ]
22 | __version__ = "0.2.0"

// ===== 文件: aoi_engine/calibration.py =====
23 | """Core Engine: 标定与量测
24 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
25 | """
26 | import numpy as np
27 |
28 | class CalibrationEngine:
29 |     """像素标定 + 尺寸量测"""
30 |
31 |     @staticmethod
32 |     def calibrate(pixel_distance: float, real_mm: float) -> float:
33 |         """从已知距离计算 pixels_per_mm"""
34 |         if real_mm <= 0: return 0
35 |         return pixel_distance / real_mm
36 |
37 |     @staticmethod
38 |     def measure(pixel_value: float, pixels_per_mm: float) -> dict:
39 |         """像素值转真实尺寸"""
40 |         if pixels_per_mm <= 0: return {"pixel": pixel_value, "real_mm": 0}
41 |         return {"pixel": round(pixel_value, 2), "real_mm": round(pixel_value /
pixels_per_mm, 4)}

// ===== 文件: aoi_engine/camera.py =====
42 | """Core Engine: 相机抽象
43 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
44 | """
45 | from abc import ABC, abstractmethod
46 | from dataclasses import dataclass
47 | import numpy as np
48 |
49 | @dataclass
50 | class CameraProperties:

// ===== Page 1 =====
```

```

51 |         width: int = 1920; height: int = 1080; fps: float = 30.0
52 |         brightness: int = 128; contrast: int = 128; exposure: int = -1
53 |         is_open: bool = False; model_name: str = "unknown"
54 |
55 | class BaseCamera(ABC):
56 |     @abstractmethod
57 |     def open(self, index: int = 0) -> bool: ...
58 |     @abstractmethod
59 |     def read_frame(self) -> np.ndarray | None: ...
60 |     @abstractmethod
61 |     def release(self) -> None: ...
62 |     @abstractmethod
63 |     def get_properties(self) -> CameraProperties: ...
64 |     def set_property(self, name: str, value: int) -> bool: return False
65 |     def __enter__(self): self.open(); return self
66 |     def __exit__(self, *a): self.release()

// ===== 文件: aoi_engine/defect.py =====
67 | """Core Engine: 缺陷检测
68 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
69 | 描述: 8 种 OasysCoreCV 检测模式, 独立于业务层.
70 | """
71 | import cv2
72 | import numpy as np
73 | from dataclasses import dataclass, field
74 |
75 | @dataclass
76 | class Defect:
77 |     type: str = ""
78 |     severity: str = "minor" # critical / major / minor
79 |     x: int = 0; y: int = 0; w: int = 0; h: int = 0
80 |     area: int = 0; description: str = ""
81 |
82 | @dataclass
83 | class InspectionResult:
84 |     rule_id: str = ""
85 |     defect_count: int = 0
86 |     pass_fail: bool = True
87 |     defects: list[Defect] = field(default_factory=list)
88 |     confidence: float = 1.0
89 |     diff_image: np.ndarray | None = None
90 |
91 | class DefectEngine:
92 |     """AOI 缺陷检测引擎 - 8 种模式"""
93 |
94 |     # === 模板比对 ===
95 |     @staticmethod
96 |     def template_match(test_frame: np.ndarray, golden_frame: np.ndarray,
threshold: float = 0.95) -> InspectionResult:
97 |         gray_t = cv2.cvtColor(test_frame, cv2.COLOR_BGR2GRAY)
98 |         gray_g = cv2.cvtColor(golden_frame, cv2.COLOR_BGR2GRAY)
99 |         diff = cv2.absdiff(gray_t, gray_g)
100 |         _, binary = cv2.threshold(diff, int((1 - threshold) * 255), 255,
cv2.THRESH_BINARY)

// ===== Page 2 =====

101 |         kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
102 |         cleaned = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
103 |         contours, _ = cv2.findContours(cleaned, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
104 |         defects = []
105 |         for c in contours:
106 |             area = cv2.contourArea(c)
107 |             if area < 50: continue
108 |             x, y, w, h = cv2.boundingRect(c)

```

```

109 |         sev = "critical" if area > 5000 else "major" if area > 1000 else
"minor"
110 |         defects.append(Defect(type="diff", severity=sev, x=int(x),
y=int(y), w=int(w), h=int(h), area=int(area), description=f"差异区域 ({int(area)}px²)"))
111 |         return InspectionResult(defect_count=len(defects),
pass_fail=len(defects) == 0, defects=defects, confidence=round(1 - len(defects) * 0.05,
2), diff_image=diff)
112 |
113 |     # === 特征匹配 (SIFT/ORB) ===
114 |     @staticmethod
115 |     def feature_match(test_frame: np.ndarray, golden_frame: np.ndarray,
min_matches: int = 10) -> InspectionResult:
116 |         gray_t = cv2.cvtColor(test_frame, cv2.COLOR_BGR2GRAY)
117 |         gray_g = cv2.cvtColor(golden_frame, cv2.COLOR_BGR2GRAY)
118 |         try: sift = cv2.SIFT_create()
119 |         except: sift = cv2.ORB_create(500)
120 |         kp1, des1 = sift.detectAndCompute(gray_g, None)
121 |         kp2, des2 = sift.detectAndCompute(gray_t, None)
122 |         if des1 is None or des2 is None:
123 |             return InspectionResult(defect_count=0, pass_fail=True,
confidence=0.0)
124 |         bf = cv2.BFMatcher() if hasattr(cv2, 'SIFT_create') else
cv2.BFMatcher(cv2.NORM_HAMMING)
125 |         matches = bf.knnMatch(des1, des2, k=2)
126 |         good = [m for m, n in matches if m.distance < 0.75 * n.distance]
127 |         missing = max(0, len(kp1) - len(good))
128 |         passed = len(good) >= min_matches
129 |         defects = []
130 |         if missing > 0:
131 |             defects.append(Defect(type="feature_missing", severity="major" if
missing > 5 else "minor", area=missing, description=f"缺失 {missing} 个特征点"))
132 |         return InspectionResult(defect_count=len(defects), pass_fail=passed,
defects=defects, confidence=round(len(good) / max(len(kp1), 1), 2))
133 |
134 |     # === 频域缺陷检测 ===
135 |     @staticmethod
136 |     def frequency_domain(test_frame: np.ndarray, threshold: float = 0.1) ->
InspectionResult:
137 |         gray = cv2.cvtColor(test_frame,
cv2.COLOR_BGR2GRAY).astype(np.float32) / 255.0
138 |         dft = cv2.dft(gray, flags=cv2.DFT_COMPLEX_OUTPUT)
139 |         dft_shift = np.fft.fftshift(dft)
140 |         magnitude = cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1])
141 |         mask = np.ones_like(magnitude)
142 |         h, w = magnitude.shape
143 |         cv2.circle(mask, (w // 2, h // 2), 30, 0, -1) # 保留低频
144 |         masked = magnitude * mask
145 |         anomaly_mask = (masked > threshold).astype(np.uint8) * 255
146 |         contours, _ = cv2.findContours(anomaly_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
147 |         defects = []
148 |         for c in contours:
149 |             area = cv2.contourArea(c)
150 |             if area < 20: continue

// ===== Page 3 =====

151 |         x, y, w, h = cv2.boundingRect(c)
152 |         defects.append(Defect(type="frequency_anomaly", severity="major",
x=int(x), y=int(y), w=int(w), h=int(h), area=int(area), description=f"频域异常
({int(area)}px²)"))
153 |         return InspectionResult(defect_count=len(defects),
pass_fail=len(defects) == 0, defects=defects, confidence=round(1 - len(defects) * 0.08,
2))
154 |
155 |     # === 颜色阈值 (HSV) ===
156 |     @staticmethod

```

```

157 |     def color_threshold(test_frame: np.ndarray, lower: tuple, upper: tuple) ->
InspectionResult:
158 |         hsv = cv2.cvtColor(test_frame, cv2.COLOR_BGR2HSV)
159 |         mask = cv2.inRange(hsv, lower, upper)
160 |         kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
161 |         cleaned = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=1)
162 |         contours, _ = cv2.findContours(cleaned, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
163 |         defects = []
164 |         for c in contours:
165 |             area = cv2.contourArea(c)
166 |             if area < 30: continue
167 |             x, y, w, h = cv2.boundingRect(c)
168 |             defects.append(Defect(type="color_deviation", severity="major" if
area > 2000 else "minor", x=int(x), y=int(y), w=int(w), h=int(h), area=int(area),
description=f"色差区域 ({int(area)}px²)")
169 |         return InspectionResult(defect_count=len(defects),
pass_fail=len(defects) == 0, defects=defects)
170 |
171 |     # === 形态学 (顶帽变换) ===
172 |     @staticmethod
173 |     def morphology_tophat(test_frame: np.ndarray, kernel_size: int = 15) ->
InspectionResult:
174 |         gray = cv2.cvtColor(test_frame, cv2.COLOR_BGR2GRAY)
175 |         kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (kernel_size,
kernel_size))
176 |         tophat = cv2.morphologyEx(gray, cv2.MORPH_TOPHAT, kernel)
177 |         _, binary = cv2.threshold(tophat, 30, 255, cv2.THRESH_BINARY)
178 |         contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
179 |         defects = []
180 |         for c in contours:
181 |             area = cv2.contourArea(c)
182 |             if area < 10: continue
183 |             x, y, w, h = cv2.boundingRect(c)
184 |             defects.append(Defect(type="tophat", severity="minor", x=int(x),
y=int(y), w=int(w), h=int(h), area=int(area), description=f"微小凸起/划痕 ({int(area)}
px²)")
185 |         return InspectionResult(defect_count=len(defects),
pass_fail=len(defects) == 0, defects=defects)
186 |
187 |     # === 轮廓分析 ===
188 |     @staticmethod
189 |     def contour_analysis(test_frame: np.ndarray, min_area: int = 100, max_area:
int = 100000) -> InspectionResult:
190 |         gray = cv2.cvtColor(test_frame, cv2.COLOR_BGR2GRAY)
191 |         blurred = cv2.GaussianBlur(gray, (5, 5), 0)
192 |         edges = cv2.Canny(blurred, 50, 150)
193 |         contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
194 |         defects = []
195 |         for c in contours:
196 |             area = cv2.contourArea(c)
197 |             if area < min_area or area > max_area: continue
198 |             x, y, w, h = cv2.boundingRect(c)
199 |             defects.append(Defect(type="contour_anomaly", severity="minor",
x=int(x), y=int(y), w=int(w), h=int(h), area=int(area), description=f"轮廓异常
({int(area)}px²)")
200 |         return InspectionResult(defect_count=len(defects),
pass_fail=len(defects) == 0, defects=defects)

```

// ===== Page 4 =====

```

201 |
202 |     # === 边缘亚像素测量 ===
203 |     @staticmethod
204 |     def subpixel_edge(frame: np.ndarray, roi: tuple) -> list:

```

```

205 |         x, y, w, h = roi
206 |         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
207 |         crop = gray[y:y + h, x:x + w]
208 |         edges = cv2.Canny(crop, 50, 150)
209 |         corners = cv2.goodFeaturesToTrack(edges, 50, 0.01, 10)
210 |         if corners is None: return []
211 |         criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30,
0.001)
212 |         refined = cv2.cornerSubPix(crop, corners, (5, 5), (-1, -1), criteria)
213 |         return [{"x": float(p[0][0]) + x, "y": float(p[0][1]) + y} for p in
refined]
214 |
215 |     # === 统一入口 ===
216 |     @staticmethod
217 |     def run(frame: np.ndarray, rule: dict, golden_frame: np.ndarray | None =
None) -> InspectionResult:
218 |         mode = rule.get("type", "template_match")
219 |         params = rule.get("params", {})
220 |         th = rule.get("threshold", 0.95)
221 |         if mode == "template_match" and golden_frame is not None:
222 |             return DefectEngine.template_match(frame, golden_frame, th)
223 |         elif mode == "feature_match" and golden_frame is not None:
224 |             return DefectEngine.feature_match(frame, golden_frame,
params.get("min_matches", 10))
225 |         elif mode == "frequency":
226 |             return DefectEngine.frequency_domain(frame, th)
227 |         elif mode == "color_threshold":
228 |             lower = tuple(params.get("lower", [0, 0, 0]))
229 |             upper = tuple(params.get("upper", [180, 255, 255]))
230 |             return DefectEngine.color_threshold(frame, lower, upper)
231 |         elif mode == "tophat":
232 |             return DefectEngine.morphology_tophat(frame,
params.get("kernel_size", 15))
233 |         elif mode == "contour":
234 |             return DefectEngine.contour_analysis(frame, params.get("min_area",
100), params.get("max_area", 100000))
235 |         return DefectEngine.template_match(frame, golden_frame, th) if
golden_frame is not None else InspectionResult()

// ===== 文件: aoi_engine/registry.py =====
236 | """Core Engine: 模块注册表 - 管理所有 OasysCoreCV 模块的启用/禁用
237 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
238 | 描述: 零依赖纯 Python 注册表。业务层通过 API 控制开关。
239 |         发布时 Cython 编译 .so - 即使服务器被黑也看不到算法逻辑。
240 | """
241 | from dataclasses import dataclass, field
242 | from typing import Callable, Any
243 |
244 |
245 | @dataclass
246 | class EngineModule:
247 |     """单个检测模块定义"""
248 |     key: str                                # 唯一标识 "template_match" /
"sift_feature" ...
249 |     name: str                              # 中文名 "模板比对"
250 |     category: str                          # 分类: detection / measurement /
vision / analysis

// ===== Page 5 =====

251 |     description: str                       # 功能描述
252 |     requires: list[str] = field(default_factory=list) # 前置依赖模块
253 |     enabled: bool = True                   # 是否启用
254 |     params: dict = field(default_factory=dict) # 模块参数
255 |     level: str = "basic"                  # basic / advanced / enterprise
256 |
257 |

```

```

258 | # ===== 全功能模块定义 =====
259 | ALL_MODULES: list[EngineModule] = [
260 |     # — 检测类 (Detection) —
261 |     EngineModule(key="template_match", name="模板比对", category="detection",
262 |         description="将待测图像与标准模板逐像素比对, 检出差异区域。最简单最快的检测方
式。",
263 |         level="basic"),
264 |     EngineModule(key="feature_match", name="特征匹配 (SIFT/ORB)",
category="detection",
265 |         description="SIFT/ORB/AKAZE 特征点匹配。旋转/缩放/光照不变, 适合定位和缺陷检
出。",
266 |         level="advanced"),
267 |     EngineModule(key="contour_analysis", name="轮廓分析", category="detection",
268 |         description="Canny边缘检测+轮廓提取。检查产品外形尺寸/边缘完整性。",
269 |         level="basic"),
270 |     EngineModule(key="color_threshold", name="颜色阈值 (HSV)",
category="detection",
271 |         description="HSV色彩空间阈值分割。检出特定颜色区域的异常 (污渍/氧化/漏印)。",
272 |         level="basic"),
273 |     EngineModule(key="frequency_domain", name="频域分析 (FFT)",
category="detection",
274 |         description="DFT/FFT 频域缺陷检测。周期性纹理缺陷 (PCB线路/布料/屏幕像素) 的最强
手段, 频域异常直接暴露。",
275 |         level="enterprise"),
276 |     EngineModule(key="morphology_tophat", name="形态学顶帽",
category="detection",
277 |         description="不均匀光照下的微小颗粒/划痕检出。比直接二值化对光照变化鲁棒得多。",
278 |         level="advanced"),
279 |
280 |     # — 量测类 (Measurement) —
281 |     EngineModule(key="subpixel_edge", name="亚像素边缘定位",
category="measurement",
282 |         description="cornerSubPix 亚像素精度角点/边缘定位。μm级精密量测的核心技术。",
283 |         level="enterprise"),
284 |     EngineModule(key="camera_calibration", name="相机标定与畸变校正",
category="measurement",
285 |         description="棋盘格标定 → 内参矩阵 + 畸变系数 → 真实世界坐标换算。消除镜头畸变误
差。",
286 |         requires=["subpixel_edge"], level="enterprise"),
287 |     EngineModule(key="stereo_vision", name="立体视觉 (Stereo)",
category="measurement",
288 |         description="双相机3D重建。BGA焊点高度/元器件共面度/3D尺寸测量。",
289 |         requires=["camera_calibration"], level="enterprise"),
290 |
291 |     # — 视觉增强 (Vision) —
292 |     EngineModule(key="image_registration", name="图像配准 (ECC)",
category="vision",
293 |         description="ECC/单应性变换。多相机对齐、抖动补偿、多图叠加去噪。",
294 |         level="advanced"),
295 |     EngineModule(key="optical_flow", name="光流追踪", category="vision",
296 |         description="Farneback/Lucas-Kanade。运动模糊补偿、传送带物体追踪、速度测
量。",
297 |         level="advanced"),
298 |     EngineModule(key="super_resolution", name="超分辨率增强", category="vision",
299 |         description="DNN超分模型。低分辨率图像增强细节。对微小缺陷检测有显著提升。",
300 |         requires=["template_match"], level="enterprise"),

```

// ===== Page 6 =====

```

301 |
302 |     # — DNN 推理 —
303 |     EngineModule(key="dnn_inference", name="DNN 深度学习推理", category="vision",
304 |         description="OasysCoreCV DNN 模块加载 ONNX/TensorFlow/PyTorch 模型。YOLO/
MobileNet/ResNet 端侧推理。",
305 |         level="enterprise"),
306 |     EngineModule(key="object_counting", name="目标计数", category="vision",
307 |         description="Blob分析+轮廓计数。零件计数、焊点计数、颗粒计数。",

```

```

308 |         level="basic"),
309 |     EngineModule(key="barcode_reader", name="条码/二维码识别", category="vision",
310 |         description="pyzbar 一维码/二维码读取。产品追溯、批次管理。",
311 |         level="basic"),
312 |
313 |     # — 拼接 —
314 |     EngineModule(key="image_stitching", name="全景拼接", category="vision",
315 |         description="OasysCoreCV Stitcher。大尺寸PCB/晶圆多图拼接。超大工件检测必
316 |         level="enterprise"),
317 | ]
318 |
319 |
320 | class ModuleRegistry:
321 |     """OasysCoreCV 模块注册表 - 单例
322 |
323 |     Usage:
324 |     reg = ModuleRegistry()
325 |     reg.enable("feature_match")    # 按客户需求启用
326 |     reg.disable("stereo_vision")   # 按客户预算禁用
327 |     reg.enabled_modules             # 获取当前启用的模块列表
328 |     """
329 |     _instance = None
330 |
331 |     def __new__(cls):
332 |         if cls._instance is None:
333 |             cls._instance = super().__new__(cls)
334 |             cls._instance._modules: dict[str, EngineModule] = {}
335 |             cls._instance._initialized = False
336 |             return cls._instance
337 |
338 |     def initialize(self, overrides: dict[str, bool] | None = None):
339 |         """启动时调用，加载所有模块。overrides 可覆盖默认启用状态。"""
340 |         self._modules = {m.key: m for m in ALL_MODULES}
341 |         if overrides:
342 |             for key, enabled in overrides.items():
343 |                 if key in self._modules:
344 |                     self._modules[key].enabled = enabled
345 |             self._initialized = True
346 |
347 |     @property
348 |     def all_modules(self) -> list[EngineModule]:
349 |         return list(self._modules.values())
350 |

```

// ===== Page 7 =====

```

351 |     @property
352 |     def enabled_modules(self) -> list[EngineModule]:
353 |         return [m for m in self._modules.values() if m.enabled]
354 |
355 |     @property
356 |     def enabled_keys(self) -> list[str]:
357 |         return [m.key for m in self.enabled_modules]
358 |
359 |     def get(self, key: str) -> EngineModule | None:
360 |         return self._modules.get(key)
361 |
362 |     def enable(self, key: str) -> bool:
363 |         m = self._modules.get(key)
364 |         if m:
365 |             # Check dependencies
366 |             for dep in m.requires:
367 |                 dep_module = self._modules.get(dep)
368 |                 if dep_module and not dep_module.enabled:
369 |                     return False # Dependency not satisfied
370 |             m.enabled = True

```

```

371 |         return True
372 |     return False
373 |
374 |     def disable(self, key: str) -> bool:
375 |         m = self._modules.get(key)
376 |         if m:
377 |             m.enabled = False
378 |             return True
379 |         return False
380 |
381 |     def toggle(self, key: str, enabled: bool) -> bool:
382 |         return self.enable(key) if enabled else self.disable(key)
383 |
384 |     def get_config(self) -> dict:
385 |         """导出当前配置（可用于保存/恢复）"""
386 |         return {
387 |             "modules": {m.key: m.enabled for m in self.all_modules},
388 |             "enabled_count": len(self.enabled_modules),
389 |             "total_count": len(self.all_modules),
390 |             "details": [
391 |                 {"key": m.key, "name": m.name, "category": m.category,
392 |                  "enabled": m.enabled, "level": m.level} for m in
self.all_modules
393 |             ]
394 |         }
395 |
396 |     def to_dict(self) -> dict:
397 |         return self.get_config()
398 |
399 |
400 | # 全局单例

// ===== Page 8 =====

401 | registry = ModuleRegistry()
402 | registry.initialize() # 默认全部启用

// ===== 文件: aoi_engine/setup.py =====
403 | """Core Engine: setup.py - Cython 编译支持
404 | =====
405 | 生产部署:
406 |     python setup.py build_ext --inplace
407 |     → 生成 aoi_engine/defect.cpython-*.so (不可逆, 不可读)
408 | """
409 | from setuptools import setup, Extension
410 | from Cython.Build import cythonize
411 | import numpy
412 |
413 | extensions = [
414 |     Extension("aoi_engine.defect", ["aoi_engine/defect.py"]),
415 |     Extension("aoi_engine.vision", ["aoi_engine/vision.py"]),
416 |     Extension("aoi_engine.calibration", ["aoi_engine/calibration.py"]),
417 | ]
418 |
419 | setup(
420 |     name="aoi-engine",
421 |     version="0.2.0",
422 |     ext_modules=cythonize(extensions, compiler_directives={"language_level":
"3"}),
423 |     include_dirs=[numpy.get_include()],
424 |     zip_safe=False,
425 | )

// ===== 文件: aoi_engine/vision.py =====
426 | """Core Engine: 视觉分析 (计数+条码+量测)
427 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
428 | """

```



```

429 | import cv2
430 | import numpy as np
431 |
432 | class VisionAnalyzer:
433 |     """计数 + 条码 + 量测"""
434 |
435 |     @staticmethod
436 |     def count_objects(frame: np.ndarray, min_area: int = 100, max_area: int =
100000, threshold: int = 127) -> dict:
437 |         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
438 |         blurred = cv2.GaussianBlur(gray, (5, 5), 0)
439 |         _, binary = cv2.threshold(blurred, threshold, 255,
cv2.THRESH_BINARY_INV)
440 |         kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
441 |         cleaned = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
442 |         contours, _ = cv2.findContours(cleaned, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
443 |         objects = []
444 |         for c in contours:
445 |             area = cv2.contourArea(c)
446 |             if min_area <= area <= max_area:
447 |                 x, y, w, h = cv2.boundingRect(c)
448 |                 objects.append({"x": int(x), "y": int(y), "w": int(w), "h":
int(h), "area": int(area)})
449 |         return {"count": len(objects), "objects": objects}
450 |

```

// ===== Page 9 =====

```

451 |     @staticmethod
452 |     def read_barcode(frame: np.ndarray) -> list[dict] | None:
453 |         try:
454 |             from pyzbar.pyzbar import decode
455 |             results = decode(frame)
456 |             return [{"data": r.data.decode("utf-8"), "type": r.type, "rect":
{"x": r.rect.left, "y": r.rect.top, "w": r.rect.width, "h": r.rect.height}} for r in
results] or None
457 |         except ImportError:
458 |             return None
459 |
460 |     @staticmethod
461 |     def measure_distance(p1: tuple, p2: tuple, pixels_per_mm: float) -> dict:
462 |         pixel_dist = np.sqrt((p2[0] - p1[0]) ** 2 + (p2[1] - p1[1]) ** 2)
463 |         return {"pixel_distance": round(pixel_dist, 2), "real_mm":
round(pixel_dist / pixels_per_mm, 3)}
464 |
465 |     @staticmethod
466 |     def measure_circle(center: tuple, radius: int, pixels_per_mm: float) ->
dict:
467 |         d_px = radius * 2
468 |         return {"diameter_px": round(d_px, 1), "diameter_mm": round(d_px /
pixels_per_mm, 3),
469 |                 "radius_mm": round(radius / pixels_per_mm, 3)}

```

// ===== 文件: backend/alembic/env.py =====

```

470 | import asyncio
471 | from logging.config import fileConfig
472 |
473 | from sqlalchemy import pool
474 | from sqlalchemy.engine import Connection
475 | from sqlalchemy.ext.asyncio import async_engine_from_config
476 |
477 | from alembic import context
478 |
479 | # this is the Alembic Config object, which provides
480 | # access to the values within the .ini file in use.
481 | config = context.config

```

```

482 |
483 | # Interpret the config file for Python logging.
484 | # This line sets up loggers basically.
485 | if config.config_file_name is not None:
486 |     fileConfig(config.config_file_name)
487 |
488 | # add your model's MetaData object here
489 | # for 'autogenerate' support
490 | from app.core.database import Base
491 | from app.modules.auth.models import User, AccountGroup, Permission # noqa:
F401
492 | from app.modules.admin.models import FieldOption # noqa: F401
493 | from app.modules.admin.audit_model import AuditLog # noqa: F401
494 | from app.modules.defects.models import InspectionRule, InspectionResult #
noqa: F401
495 | from app.modules.measurement.models import Calibration, MeasurementRecord #
noqa: F401
496 | target_metadata = Base.metadata
497 |
498 | # other values from the config, defined by the needs of env.py,
499 | # can be acquired:
500 | # my_important_option = config.get_main_option("my_important_option")

```

// ===== Page 10 =====

```

501 | # ... etc.
502 |
503 |
504 | def run_migrations_offline() -> None:
505 |     """Run migrations in 'offline' mode.
506 |
507 |     This configures the context with just a URL
508 |     and not an Engine, though an Engine is acceptable
509 |     here as well. By skipping the Engine creation
510 |     we don't even need a DBAPI to be available.
511 |
512 |     Calls to context.execute() here emit the given string to the
513 |     script output.
514 |
515 |     """
516 |     url = config.get_main_option("sqlalchemy.url")
517 |     context.configure(
518 |         url=url,
519 |         target_metadata=target_metadata,
520 |         literal_binds=True,
521 |         dialect_opts={"paramstyle": "named"},
522 |     )
523 |
524 |     with context.begin_transaction():
525 |         context.run_migrations()
526 |
527 |
528 | def do_run_migrations(connection: Connection) -> None:
529 |     context.configure(connection=connection, target_metadata=target_metadata)
530 |
531 |     with context.begin_transaction():
532 |         context.run_migrations()
533 |
534 |
535 | async def run_async_migrations() -> None:
536 |     """In this scenario we need to create an Engine
537 |     and associate a connection with the context.
538 |
539 |     """
540 |
541 |     connectable = async_engine_from_config(
542 |         config.get_section(config.config_ini_section, {}),

```

```

543 |         prefix="sqlalchemy.",
544 |         poolclass=pool.NullPool,
545 |     )
546 |
547 |     async with connectable.connect() as connection:
548 |         await connection.run_sync(do_run_migrations)
549 |
550 |     await connectable.dispose()

```

// ===== Page 11 =====

```

551 |
552 |
553 | def run_migrations_online() -> None:
554 |     """Run migrations in 'online' mode."""
555 |
556 |     asyncio.run(run_async_migrations())
557 |
558 |
559 | if context.is_offline_mode():
560 |     run_migrations_offline()
561 | else:
562 |     run_migrations_online()

```

// ===== 文件: backend/app/__init__.py =====

```

563 | """AOI-Vision v0.1
564 | """
565 | from app.core.database import Base
566 | from app.core.config import settings

```

// ===== 文件: backend/app/main.py =====

```

567 | """AOI-Vision v0.1 – 自动光学检测系统
568 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
569 | 描述: FastAPI 应用入口, 路由注册
570 | """
571 | from contextlib import asynccontextmanager
572 | import sys, os
573 | # Add project root to sys.path so aoi_engine can be imported
574 | _project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
575 | # __file__ = backend/app/main.py → backend/app → backend → project root
576 | sys.path.insert(0, os.path.dirname(_project_root)) # parent of backend/ = AOI-
Vision/
577 | from fastapi import FastAPI
578 | from fastapi.middleware.cors import CORSMiddleware
579 | from fastapi.staticfiles import StaticFiles
580 | from app.core.database import init_db
581 | from app.modules.auth import router as auth_router
582 | from app.modules.admin import router as admin_router
583 | from app.modules.defects import router as defects_router
584 | from app.modules.measurement import router as measurement_router
585 | from app.modules.reports import router as reports_router
586 | from app.modules.device_router import router as device_router
587 | from app.modules.inspection.websocket import router as ws_router
588 |
589 | @asynccontextmanager
590 | async def lifespan(app: FastAPI):
591 |     await init_db()
592 |     yield
593 |
594 | app = FastAPI(title="AOI-Vision", version="0.1.0", lifespan=lifespan)
595 | app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])
596 | app.include_router(auth_router)
597 | app.include_router(admin_router)
598 | app.include_router(defects_router)
599 | app.include_router(measurement_router)
600 | app.include_router(reports_router)

```

```
// ===== Page 12 =====
```

```
601 | app.include_router(device_router)
602 | app.include_router(ws_router)
603 |
604 | import os
605 | static_path = os.path.join(os.path.dirname(__file__), "../static")
606 | if os.path.exists(static_path):
607 |     app.mount("/app", StaticFiles(directory=static_path, html=True),
name="static")
608 |
609 | @app.get("/health")
610 | async def health():
611 |     return {"status": "ok", "version": "0.1.0"}
612 |
613 | @app.get("/")
614 | async def root():
615 |     from fastapi.responses import RedirectResponse
616 |     return RedirectResponse(url="/app/")
```

```
// ===== 文件: backend/app/core/__init__.py =====
```

```
617 | """AOI-Vision v0.1
618 | """
```

```
// ===== 文件: backend/app/core/config.py =====
```

```
619 | """AOI-Vision v0.1
620 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
621 | """
622 | from pydantic_settings import BaseSettings
623 |
624 | class Settings(BaseSettings):
625 |     DATABASE_URL: str = "sqlite+aiosqlite:///./aoi_vision.db"
626 |     JWT_SECRET_KEY: str = "change-me-in-production"
627 |     JWT_ACCESS_TOKEN_EXPIRE_MINUTES: int = 60
628 |     UPLOAD_DIR: str = "./uploads"
629 |     CAMERA_INDEX: int = 0
630 |     CAMERA_TYPE: str = "mock" # mock | uvc | gige | hikvision | ...
631 |     DEVICE_GATEWAY_TYPE: str = "mock" # mock | haas506 | modbus | opcua | ...
632 |
633 |     class Config:
634 |         env_file = ".env"
635 |
636 | settings = Settings()
```

```
// ===== 文件: backend/app/core/database.py =====
```

```
637 | """AOI-Vision v0.1
638 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
639 | 描述: SQLAlchemy 异步引擎 + SQLite/PostgreSQL 双模
640 | """
641 | import uuid as _uuid
642 | from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker,
create_async_engine
643 | from sqlalchemy.orm import DeclarativeBase
644 | from sqlalchemy.types import TypeDecorator, CHAR
645 | from app.core.config import settings
646 |
647 | class GUID(TypeDecorator):
648 |     impl = CHAR; cache_ok = True
649 |     def load_dialect_impl(self, dialect):
650 |         return dialect.type_descriptor(CHAR(36)) if dialect.name == "sqlite"
else dialect.type_descriptor(self.impl)
```

```
// ===== Page 13 =====
```

```
651 |     def process_bind_param(self, value, dialect):
652 |         if value is None: return value
```

```

653 |         return str(value) if dialect.name == "sqlite" else value
654 |     def process_result_value(self, value, dialect):
655 |         if value is None: return value
656 |         return _uuid.UUID(value) if not isinstance(value, _uuid.UUID) else
value
657 |
658 |     if "sqlite" in settings.DATABASE_URL:
659 |         import sqlalchemy.dialects.sqlite as _sq
660 |         _sq.base.ischema_names["uuid"] = GUID
661 |
662 |     engine = create_async_engine(settings.DATABASE_URL, echo=False)
663 |     async_session = async_sessionmaker(engine, class_=AsyncSession,
expire_on_commit=False)
664 |
665 |     if "sqlite" in settings.DATABASE_URL:
666 |         from sqlalchemy import event
667 |         @event.listens_for(engine.sync_engine, "connect")
668 |         def _sqlite_fk(dbapi_connection, connection_record):
669 |             dbapi_connection.execute("PRAGMA foreign_keys=ON")
670 |
671 |     class Base(DeclarativeBase): pass
672 |
673 |     async def get_db():
674 |         async with async_session() as session:
675 |             try:
676 |                 yield session
677 |                 await session.commit()
678 |             except Exception:
679 |                 await session.rollback()
680 |             raise
681 |
682 |     async def init_db():
683 |         """创建表 + 种子数据"""
684 |         async with engine.begin() as conn:
685 |             await conn.run_sync(Base.metadata.create_all)
686 |
687 |         from app.modules.admin.service import seed_default_options
688 |         from app.modules.auth.service import seed_all
689 |
690 |         async with async_session() as db:
691 |             await seed_default_options(db)
692 |             await seed_all(db)
693 |             await db.commit()

```

// ===== 文件: backend/app/core/exceptions.py =====

```

694 | """AOI-Vision v0.1
695 | """
696 | from fastapi import HTTPException, status
697 |
698 | class AppException(HTTPException):
699 |     def __init__(self, status_code: int, detail: str):
700 |         super().__init__(status_code=status_code, detail=detail)

```

// ===== Page 14 =====

// ===== 文件: backend/app/core/security.py =====

```

701 | """AOI-Vision v0.1 - 安全模块
702 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
703 | 描述: JWT 令牌生成/验证 + 密码哈希 (bcrypt)
704 | """
705 | from datetime import datetime, timedelta, timezone
706 | from jose import JWTError, jwt
707 | import bcrypt
708 | from app.core.config import settings
709 |
710 | def hash_password(password: str) -> str:

```

```

711 |     return bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()
712 |
713 | def verify_password(plain: str, hashed: str) -> bool:
714 |     return bcrypt.checkpw(plain.encode(), hashed.encode())
715 |
716 | def create_access_token(data: dict, expires_minutes: int | None = None) -> str:
717 |     expire = datetime.now(timezone.utc) + timedelta(minutes=expires_minutes or
settings.JWT_ACCESS_TOKEN_EXPIRE_MINUTES)
718 |     to_encode = data.copy()
719 |     to_encode.update({"exp": expire})
720 |     return jwt.encode(to_encode, settings.JWT_SECRET_KEY, algorithm="HS256")
721 |
722 | def decode_access_token(token: str) -> dict | None:
723 |     try:
724 |         return jwt.decode(token, settings.JWT_SECRET_KEY, algorithms=["HS256"])
725 |     except JWTError:
726 |         return None

```

// ===== 文件: backend/app/hardware/__init__.py =====

```

727 | """AOI-Vision v0.1 - 硬件层
728 | """
729 | from app.hardware.camera import BaseCamera, CameraProperties
730 | from app.hardware.device import BaseDeviceGateway, DeviceTelemetry
731 | from app.hardware.factory import create_camera, create_device_gateway,
CAMERA_REGISTRY, DEVICE_REGISTRY
732 |
733 | __all__ = [
734 |     "BaseCamera", "CameraProperties",
735 |     "BaseDeviceGateway", "DeviceTelemetry",
736 |     "create_camera", "create_device_gateway",
737 |     "CAMERA_REGISTRY", "DEVICE_REGISTRY",
738 | ]

```

// ===== 文件: backend/app/hardware/camera.py =====

```

739 | """AOI-Vision v0.1 - 硬件抽象层
740 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
741 | 描述: 摄像头抽象接口 - 所有相机实现必须遵循此协议。
742 |       换硬件只需新增一个子类, 不改业务代码。
743 | """
744 | from abc import ABC, abstractmethod
745 | from dataclasses import dataclass
746 | from typing import Optional
747 | import numpy as np
748 |
749 |
750 | @dataclass

```

// ===== Page 15 =====

```

751 | class CameraProperties:
752 |     """相机属性快照"""
753 |     width: int = 1920
754 |     height: int = 1080
755 |     fps: float = 30.0
756 |     brightness: int = 128
757 |     contrast: int = 128
758 |     exposure: int = -1 # -1 = auto
759 |     is_open: bool = False
760 |     model_name: str = "unknown"
761 |
762 |
763 | class BaseCamera(ABC):
764 |     """摄像头抽象基类 - 所有相机驱动必须实现"""
765 |
766 |     @abstractmethod
767 |     def open(self, index: int = 0) -> bool:
768 |         """打开摄像头, 返回是否成功"""

```

```

769 |         ...
770 |
771 |     @abstractmethod
772 |     def read_frame(self) -> Optional[np.ndarray]:
773 |         """读取一帧 (BGR numpy array), 无帧返回 None"""
774 |         ...
775 |
776 |     @abstractmethod
777 |     def release(self) -> None:
778 |         """释放摄像头资源"""
779 |         ...
780 |
781 |     @abstractmethod
782 |     def get_properties(self) -> CameraProperties:
783 |         """获取当前相机属性"""
784 |         ...
785 |
786 |     def set_property(self, name: str, value: int) -> bool:
787 |         """设置相机属性 (亮度/对比度/曝光), 返回是否成功。
788 |         子类可选覆写。"""
789 |         return False
790 |
791 |     def __enter__(self):
792 |         self.open()
793 |         return self
794 |
795 |     def __exit__(self, *args):
796 |         self.release()

```

// ===== 文件: backend/app/hardware/device.py =====

```

797 | """AOI-Vision v0.1 - 设备网关抽象层
798 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
799 | 描述: 远程设备网关抽象接口。
800 |     换设备只需新增子类, 不改业务代码。

```

// ===== Page 16 =====

```

801 | """
802 | from abc import ABC, abstractmethod
803 | from dataclasses import dataclass, field
804 | from typing import Optional, Any
805 | from datetime import datetime, timezone
806 |
807 |
808 | @dataclass
809 | class DeviceTelemetry:
810 |     """设备遥测数据"""
811 |     device_id: str = ""
812 |     online: bool = False
813 |     signal_strength: int = 0          # CSQ 0-31
814 |     temperature: float = 0.0         # 设备温度 °C
815 |     uptime_seconds: int = 0
816 |     relay1_state: bool = False        # 继电器1状态
817 |     relay2_state: bool = False
818 |     gps_lat: float = 0.0
819 |     gps_lng: float = 0.0
820 |     timestamp: datetime = field(default_factory=lambda:
datetime.now(timezone.utc))
821 |
822 |
823 | class BaseDeviceGateway(ABC):
824 |     """设备网关抽象基类"""
825 |
826 |     @abstractmethod
827 |     async def connect(self) -> bool:
828 |         """连接设备, 返回是否成功"""
829 |         ...

```

```

830 |
831 |     @abstractmethod
832 |     async def get_telemetry(self) -> DeviceTelemetry:
833 |         """获取设备遥测数据"""
834 |         ...
835 |
836 |     @abstractmethod
837 |     async def send_command(self, command: str, params: dict[str, Any] | None =
None) -> dict:
838 |         """发送控制指令，返回执行结果"""
839 |         ...
840 |
841 |     @abstractmethod
842 |     async def disconnect(self) -> None:
843 |         """断开连接"""
844 |         ...

```

```

// ===== 文件: backend/app/hardware/factory.py =====
845 | """AOI-Vision v0.1 - 硬件工厂 (懒加载版)
846 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
847 | 描述: 根据配置创建相机和网关实例。
848 |       换硬件只改 config，不碰业务代码。
849 |       驱动模块懒加载—没有 OasysCoreCV 时抽象层仍可 import。
850 | """

```

```

// ===== Page 17 =====

```

```

851 | from typing import Type
852 | from app.hardware.camera import BaseCamera
853 | from app.hardware.device import BaseDeviceGateway
854 |
855 |
856 | CAMERA_REGISTRY: dict[str, str] = {
857 |     "uvc": "app.hardware.uvc_camera.UVCCamera",
858 |     "mock": "app.hardware.mock_camera.MockCamera",
859 |     # 未来扩展:
860 |     # "gige": "app.hardware.gige_camera.GigECamera",
861 |     # "hikvision": "app.hardware.hik_camera.HikCamera",
862 | }
863 |
864 | DEVICE_REGISTRY: dict[str, str] = {
865 |     "haas506": "app.hardware.haas506_gateway.HaaS506Gateway",
866 |     "mock": "app.hardware.mock_device.MockDeviceGateway",
867 |     # 未来扩展:
868 |     # "modbus": "app.hardware.modbus_gateway.ModbusGateway",
869 |     # "opcua": "app.hardware.opcua_gateway.OPCUAGateway",
870 | }
871 |
872 |
873 | def _import_class(dotted_path: str) -> Type:
874 |     """懒加载导入类"""
875 |     module_path, class_name = dotted_path.rsplit(".", 1)
876 |     import importlib
877 |     module = importlib.import_module(module_path)
878 |     return getattr(module, class_name)
879 |
880 |
881 | def create_camera(camera_type: str = "mock", **kwargs) -> BaseCamera:
882 |     """根据类型创建相机实例。
883 |
884 |     Args:
885 |         camera_type: 'uvc' | 'mock' | 'gige' | 'hikvision' ...
886 |         **kwargs: 传递给具体相机类 (如 index=0, mode='checkerboard')
887 |
888 |     Example:
889 |         cam = create_camera('mock', mode='checkerboard', width=1920,
height=1080)

```



```

890 |         cam = create_camera('uvc', index=0)
891 |         """
892 |         dotted = CAMERA_REGISTRY.get(camera_type)
893 |         if dotted is None:
894 |             raise ValueError(f"Unknown camera type: {camera_type}. Available:
{list(CAMERA_REGISTRY.keys())}")
895 |         cls = _import_class(dotted)
896 |         return cls(**kwargs)
897 |
898 |
899 | def create_device_gateway(gateway_type: str = "mock", **kwargs) ->
BaseDeviceGateway:
900 |     """根据类型创建设备网关实例。

```

// ===== Page 18 =====

```

901 |
902 |     Args:
903 |         gateway_type: 'haas506' | 'mock' | 'modbus' | 'opcua' ...
904 |         **kwargs: device_id, product_key, ...
905 |
906 |     Example:
907 |         gw = create_device_gateway('mock', device_id='test-001')
908 |         gw = create_device_gateway('haas506', device_id='hd3-001',
product_key='...')
909 |         """
910 |         dotted = DEVICE_REGISTRY.get(gateway_type)
911 |         if dotted is None:
912 |             raise ValueError(f"Unknown device type: {gateway_type}. Available:
{list(DEVICE_REGISTRY.keys())}")
913 |         cls = _import_class(dotted)
914 |         return cls(**kwargs)

```

// ===== 文件: backend/app/hardware/haas506_gateway.py =====

```

915 | """A0I-Vision v0.1 - HaaS506 设备网关
916 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
917 | 描述: 通过 MQTT/阿里云 IoT 与 HaaS506-HD3 通信。
918 |         远程获取遥测、控制继电器、接收告警。
919 | """
920 | import json
921 | import asyncio
922 | from typing import Optional, Any
923 | from datetime import datetime, timezone
924 | from app.hardware.device import BaseDeviceGateway, DeviceTelemetry
925 |
926 |
927 | class HaaS506Gateway(BaseDeviceGateway):
928 |     """HaaS506-HD3 4G RTU 设备网关 (MQTT)"""
929 |
930 |     def __init__(self, device_id: str, product_key: str = "",
931 |                  device_name: str = "", device_secret: str = "",
932 |                  region: str = "cn-shanghai"):
933 |         self.device_id = device_id
934 |         self._product_key = product_key
935 |         self._device_name = device_name
936 |         self._device_secret = device_secret
937 |         self._region = region
938 |         self._connected = False
939 |         self._last_telemetry: Optional[DeviceTelemetry] = None
940 |
941 |     async def connect(self) -> bool:
942 |         """连接 HaaS506，通过阿里云 IoT MQTT"""
943 |         # TODO: 实际实现使用 aliyunIoT SDK 或 paho-mqtt
944 |         # 现阶段返回模拟连接状态
945 |         self._connected = True
946 |         return True
947 |

```

```

948 |         async def get_telemetry(self) -> DeviceTelemetry:
949 |             """从 HaaS506 拉取最新遥测"""
950 |             if self._last_telemetry:

// ===== Page 19 =====

951 |                 self._last_telemetry.timestamp = datetime.now(timezone.utc)
952 |                 return self._last_telemetry
953 |             # 模拟遥测
954 |             return DeviceTelemetry(
955 |                 device_id=self.device_id,
956 |                 online=self._connected,
957 |                 signal_strength=25,
958 |                 temperature=42.5,
959 |                 uptime_seconds=3600,
960 |                 relay1_state=False,
961 |                 relay2_state=False,
962 |             )
963 |
964 |         async def send_command(self, command: str, params: dict[str, Any] | None =
None) -> dict:
965 |             """通过 MQTT 下发指令到 HaaS506"""
966 |             if not self._connected:
967 |                 return {"success": False, "error": "Device not connected"}
968 |             # TODO: 实际 MQTT publish
969 |             if command == "set_relay":
970 |                 relay_id = params.get("relay", 1) if params else 1
971 |                 state = params.get("state", False) if params else False
972 |                 return {"success": True, "relay": relay_id, "state": state}
973 |             elif command == "reboot":
974 |                 return {"success": True, "action": "reboot_scheduled"}
975 |             return {"success": False, "error": f"Unknown command: {command}"}
976 |
977 |         async def disconnect(self) -> None:
978 |             self._connected = False

// ===== 文件: backend/app/hardware/mock_camera.py =====
979 | """AOI-Vision v0.1 - Mock 相机
980 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
981 | 描述: 无硬件时的模拟相机 - 用于开发测试/CI。
982 | 支持纯色图、棋盘格、从图片文件读取。
983 | """
984 | import cv2
985 | import numpy as np
986 | from typing import Optional
987 | from app.hardware.camera import BaseCamera, CameraProperties
988 |
989 |
990 | class MockCamera(BaseCamera):
991 |     """模拟相机 - 无硬件依赖, 用于开发和自动化测试"""
992 |
993 |     MODE_SOLID = "solid"
994 |     MODE_CHECKERBOARD = "checkerboard"
995 |     MODE_FILE = "file"
996 |
997 |     def __init__(self, mode: str = MODE_CHECKERBOARD,
998 |                  width: int = 1920, height: int = 1080,
999 |                  file_path: Optional[str] = None):
1000 |         self._mode = mode

// ===== Page 20 =====

1001 |         self._file_path = file_path
1002 |         self._frame_cache: Optional[np.ndarray] = None
1003 |         self._props = CameraProperties(
1004 |             width=width, height=height, fps=30,
1005 |             model_name=f"Mock Camera ({mode})"

```

```

1006 |         )
1007 |
1008 |     def open(self, index: int = 0) -> bool:
1009 |         if self._mode == self.MODE_FILE and self._file_path:
1010 |             img = cv2.imread(self._file_path)
1011 |             if img is not None:
1012 |                 self._frame_cache = cv2.resize(img, (self._props.width,
self._props.height))
1013 |             if self._mode == self.MODE_SOLID:
1014 |                 self._frame_cache = np.ones(
1015 |                     (self._props.height, self._props.width, 3), dtype=np.uint8) *
128
1016 |                 self._props.is_open = True
1017 |                 return True
1018 |
1019 |     def read_frame(self) -> Optional[np.ndarray]:
1020 |         if not self._props.is_open:
1021 |             return None
1022 |         if self._frame_cache is not None:
1023 |             return self._frame_cache.copy()
1024 |         # 动态生成棋盘格
1025 |         size = 100
1026 |         rows = self._props.height // size + 1
1027 |         cols = self._props.width // size + 1
1028 |         frame = np.zeros((self._props.height, self._props.width, 3),
dtype=np.uint8)
1029 |         for r in range(rows):
1030 |             for c in range(cols):
1031 |                 if (r + c) % 2 == 0:
1032 |                     y1, y2 = r * size, min((r + 1) * size, self._props.height)
1033 |                     x1, x2 = c * size, min((c + 1) * size, self._props.width)
1034 |                     frame[y1:y2, x1:x2] = 200
1035 |         return frame
1036 |
1037 |     def release(self) -> None:
1038 |         self._props.is_open = False
1039 |
1040 |     def get_properties(self) -> CameraProperties:
1041 |         return self._props

```

// ===== 文件: backend/app/hardware/mock_device.py =====

```

1042 | """AOI-Vision v0.1 - Mock 设备网关
1043 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
1044 | 描述: 无硬件时的模拟设备网关, 用于开发测试。
1045 | """
1046 | import random
1047 | from typing import Any
1048 | from datetime import datetime, timezone
1049 | from app.hardware.device import BaseDeviceGateway, DeviceTelemetry
1050 |

```

// ===== Page 21 =====

```

1051 |
1052 | class MockDeviceGateway(BaseDeviceGateway):
1053 |     """模拟设备网关 - 无硬件依赖"""
1054 |
1055 |     def __init__(self, device_id: str = "mock-device-001"):
1056 |         self.device_id = device_id
1057 |         self._connected = False
1058 |         self._relay1 = False
1059 |         self._relay2 = False
1060 |         self._start_time = datetime.now(timezone.utc)
1061 |
1062 |     async def connect(self) -> bool:
1063 |         self._connected = True
1064 |         return True

```

```

1065 |
1066 |     async def get_telemetry(self) -> DeviceTelemetry:
1067 |         uptime = int((datetime.now(timezone.utc) -
self._start_time).total_seconds())
1068 |         return DeviceTelemetry(
1069 |             device_id=self.device_id,
1070 |             online=self._connected,
1071 |             signal_strength=random.randint(20, 31),
1072 |             temperature=round(random.uniform(35, 50), 1),
1073 |             uptime_seconds=uptime,
1074 |             relay1_state=self._relay1,
1075 |             relay2_state=self._relay2,
1076 |             gps_lat=22.5431 + random.uniform(-0.01, 0.01),
1077 |             gps_lng=113.929 + random.uniform(-0.01, 0.01),
1078 |         )
1079 |
1080 |     async def send_command(self, command: str, params: dict[str, Any] | None =
None) -> dict:
1081 |         if not self._connected:
1082 |             return {"success": False, "error": "Not connected"}
1083 |         if command == "set_relay":
1084 |             relay_id = params.get("relay", 1) if params else 1
1085 |             state = params.get("state", False) if params else False
1086 |             if relay_id == 1:
1087 |                 self._relay1 = state
1088 |             else:
1089 |                 self._relay2 = state
1090 |             return {"success": True}
1091 |         return {"success": False, "error": f"Unknown: {command}"}
1092 |
1093 |     async def disconnect(self) -> None:
1094 |         self._connected = False

```

// ===== 文件: backend/app/hardware/uvc_camera.py =====

```

1095 | """AOI-Vision v0.1 - UVC 相机驱动
1096 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
1097 | 描述: 通用 USB Video Class 摄像头实现。
1098 |         兼容龙仕显微镜摄像头及所有标准 UVC 设备。
1099 | """
1100 | import cv2

```

// ===== Page 22 =====

```

1101 | import numpy as np
1102 | from typing import Optional
1103 | from app.hardware.camera import BaseCamera, CameraProperties
1104 |
1105 |
1106 | class UVCCamera(BaseCamera):
1107 |     """通用 USB 相机 - 基于 OasysCoreCV VideoCapture"""
1108 |
1109 |     def __init__(self):
1110 |         self._cap: Optional[cv2.VideoCapture] = None
1111 |         self._props = CameraProperties(model_name="UVC Camera")
1112 |
1113 |     def open(self, index: int = 0) -> bool:
1114 |         self._cap = cv2.VideoCapture(index)
1115 |         if not self._cap.isOpened():
1116 |             self._cap = None
1117 |             return False
1118 |         # 尝试读取实际参数
1119 |         w = int(self._cap.get(cv2.CAP_PROP_FRAME_WIDTH))
1120 |         h = int(self._cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
1121 |         fps = self._cap.get(cv2.CAP_PROP_FPS)
1122 |         if w > 0 and h > 0:
1123 |             self._props.width = w
1124 |             self._props.height = h

```

```

1125 |         if fps > 0:
1126 |             self._props.fps = fps
1127 |         self._props.is_open = True
1128 |         return True
1129 |
1130 |     def read_frame(self) -> Optional[np.ndarray]:
1131 |         if self._cap is None:
1132 |             return None
1133 |         ret, frame = self._cap.read()
1134 |         return frame if ret else None
1135 |
1136 |     def release(self) -> None:
1137 |         if self._cap:
1138 |             self._cap.release()
1139 |             self._cap = None
1140 |         self._props.is_open = False
1141 |
1142 |     def get_properties(self) -> CameraProperties:
1143 |         return self._props
1144 |
1145 |     def set_property(self, name: str, value: int) -> bool:
1146 |         if self._cap is None:
1147 |             return False
1148 |         prop_map = {
1149 |             "brightness": cv2.CAP_PROP_BRIGHTNESS,
1150 |             "contrast": cv2.CAP_PROP_CONTRAST,
1151 |             "exposure": cv2.CAP_PROP_EXPOSURE,
1152 |             "width": cv2.CAP_PROP_FRAME_WIDTH,
1153 |             "height": cv2.CAP_PROP_FRAME_HEIGHT,
1154 |         }
1155 |         prop_id = prop_map.get(name)
1156 |         if prop_id is None:
1157 |             return False
1158 |         self._cap.set(prop_id, value)
1159 |         return True
1160 |
1161 | // ===== Page 23 =====
1162 |
1163 | // ===== 文件: backend/app/modules/device_router.py =====
1164 | """AOI-Vision v0.1 - 设备网关路由
1165 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
1166 | 描述: 设备管理 API - HaaS506 遥测查看 + 远程控制
1167 | """
1168 | from fastapi import APIRouter, Depends
1169 | from app.core.config import settings
1170 | from app.modules.auth.service import get_current_user
1171 | from app.hardware import create_device_gateway
1172 |
1173 | router = APIRouter(prefix="/device", tags=["Device"])
1174 |
1175 | @router.get("/telemetry")
1176 | async def device_telemetry(_user=Depends(get_current_user)):
1177 |     gw = create_device_gateway(settings.DEVICE_GATEWAY_TYPE)
1178 |     await gw.connect()
1179 |     try:
1180 |         telemetry = await gw.get_telemetry()
1181 |         return telemetry
1182 |     finally:
1183 |         await gw.disconnect()
1184 |
1185 | @router.post("/relay")
1186 | async def set_relay(relay: int = 1, state: bool = False,
1187 | _user=Depends(get_current_user)):
1188 |     from app.modules.auth.service import require_permission
1189 |     _admin = require_permission("device", "manage")
1190 |     gw = create_device_gateway(settings.DEVICE_GATEWAY_TYPE)

```

```

1186 |     await gw.connect()
1187 |     try:
1188 |         result = await gw.send_command("set_relay", {"relay": relay, "state":
state})
1189 |         return result
1190 |     finally:
1191 |         await gw.disconnect()

```

// ===== 文件: backend/app/modules/admin/__init__.py =====

```

1192 | """AOI-Vision v0.1
1193 | """
1194 | from app.modules.admin.router import router
1195 | from app.modules.admin.service import seed_default_options
1196 |
1197 | __all__ = ["router", "seed_default_options"]

```

// ===== 文件: backend/app/modules/admin/audit_model.py =====

```

1198 | """AOI-Vision v0.1 - 审计日志模型
1199 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
1200 | 描述: ERPNext 风格审计 - CUD 操作全记录

```

// ===== Page 24 =====

```

1201 | """
1202 | import uuid
1203 | from datetime import datetime, timezone
1204 | from sqlalchemy import DateTime, String, Text, func
1205 | from app.core.database import GUID
1206 | from sqlalchemy.dialects.sqlite import JSON as SQLiteJSON
1207 | from sqlalchemy.orm import Mapped, mapped_column
1208 | from app.core.config import settings
1209 | from app.core.database import Base
1210 |
1211 | # SQLite 用 JSON, PostgreSQL 用 JSONB
1212 | JSONType = SQLiteJSON if "sqlite" in settings.DATABASE_URL else
__import__("sqlalchemy.dialects.postgresql", fromlist=["JSONB"]).JSONB
1213 |
1214 | class AuditLog(Base):
1215 |     __tablename__ = "audit_logs"
1216 |     id: Mapped[uuid.UUID] = mapped_column(GUID, primary_key=True,
default=uuid.uuid4)
1217 |     timestamp: Mapped[datetime] = mapped_column(DateTime(timezone=True),
default=lambda: datetime.now(timezone.utc), server_default=func.now(), index=True)
1218 |     username: Mapped[str] = mapped_column(String(100), nullable=False,
default="system")
1219 |     action: Mapped[str] = mapped_column(String(20), nullable=False, index=True)
1220 |     entity_type: Mapped[str] = mapped_column(String(100), nullable=False,
index=True)
1221 |     entity_id: Mapped[str | None] = mapped_column(String(100), nullable=True)
1222 |     entity_label: Mapped[str | None] = mapped_column(String(200),
nullable=True)
1223 |     changes: Mapped[dict | None] = mapped_column(JSONType, nullable=True)
1224 |     summary: Mapped[str | None] = mapped_column(Text, nullable=True)

```

// ===== 文件: backend/app/modules/admin/models.py =====

```

1225 | """AOI-Vision v0.1 - 参数枚举模块
1226 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
1227 | 描述: FieldOption 模型 - 可配置的下拉选项 (缺陷类型/产品型号/产线等)
1228 | """
1229 | from sqlalchemy import Boolean, Integer, String
1230 | from sqlalchemy.orm import Mapped, mapped_column
1231 | from app.core.database import Base
1232 |
1233 | class FieldOption(Base):
1234 |     __tablename__ = "field_options"
1235 |     id: Mapped[int] = mapped_column(Integer, primary_key=True,
autoincrement=True)

```

```
1236 |         field_name: Mapped[str] = mapped_column(String(100), nullable=False,
index=True)
1237 |         option_value: Mapped[str] = mapped_column(String(200), nullable=False)
1238 |         option_label: Mapped[str] = mapped_column(String(200), nullable=False)
1239 |         sort_order: Mapped[int] = mapped_column(Integer, default=0)
1240 |         is_active: Mapped[bool] = mapped_column(Boolean, default=True)
```

```
// ===== 文件: backend/app/modules/admin/router.py =====
```

```
1241 | """AOI-Vision v0.1 - 管理路由
1242 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
1243 | """
1244 | from fastapi import APIRouter, Depends, Query
1245 | from sqlalchemy.ext.asyncio import AsyncSession
1246 | from app.core.database import get_db
1247 | from app.modules.auth.service import get_current_user, require_permission
1248 | from app.modules.admin.schemas import FieldOptionCreate, FieldOptionUpdate,
FieldOptionResponse, AuditLogListResponse
1249 | from app.modules.admin.service import list_options, create_option,
update_option, delete_option, query_audit_logs
1250 |
```

```
// ===== Page 25 =====
```

```
1251 | router = APIRouter(prefix="/admin", tags=["Admin"])
1252 |
1253 | @router.get("/field-options", response_model=list[FieldOptionResponse])
1254 | async def get_options(field_name: str | None = Query(None), db: AsyncSession =
Depends(get_db), _user=Depends(get_current_user)):
1255 |     return await list_options(db, field_name)
1256 |
1257 | @router.post("/field-options", response_model=FieldOptionResponse)
1258 | async def add_option(data: FieldOptionCreate, db: AsyncSession =
Depends(get_db), _admin=require_permission("admin", "manage")):
1259 |     return await create_option(db, data)
1260 |
1261 | @router.put("/field-options/{opt_id}", response_model=FieldOptionResponse)
1262 | async def edit_option(opt_id: int, data: FieldOptionUpdate, db: AsyncSession =
Depends(get_db), _admin=require_permission("admin", "manage")):
1263 |     opt = await update_option(db, opt_id, data)
1264 |     if not opt:
1265 |         from app.core.exceptions import AppException
1266 |         raise AppException(404, "Option not found")
1267 |     return opt
1268 |
1269 | @router.delete("/field-options/{opt_id}", status_code=204)
1270 | async def remove_option(opt_id: int, db: AsyncSession = Depends(get_db),
_admin=require_permission("admin", "manage")):
1271 |     ok = await delete_option(db, opt_id)
1272 |     if not ok:
1273 |         from app.core.exceptions import AppException
1274 |         raise AppException(404, "Option not found")
1275 |
1276 | # ===== 模块管理 =====
1277 |
1278 | @router.get("/engine-modules")
1279 | async def get_modules(_user=Depends(get_current_user)):
1280 |     """查看所有 OasysCoreCV 模块及启用状态"""
1281 |     from aoi_engine import registry
1282 |     return registry.get_config()
1283 |
1284 | @router.post("/engine-modules/{module_key}")
1285 | async def toggle_module(module_key: str, data: dict,
_admin=require_permission("admin", "manage")):
1286 |     """启用/禁用指定模块"""
1287 |     from aoi_engine import registry
1288 |     enabled = data.get("enabled", True)
1289 |     ok = registry.toggle(module_key, enabled)
```

```

1290 |         return {"key": module_key, "enabled": registry.get(module_key).enabled if
registry.get(module_key) else None, "success": ok}
1291 |
1292 | @router.get("/engine-modules/config")
1293 | async def export_module_config(_user=Depends(get_current_user)):
1294 |     """导出当前模块配置快照（用于客户打包）"""
1295 |     from aoi_engine import registry
1296 |     return registry.get_config()
1297 |
1298 | @router.get("/audit-logs", response_model=AuditLogListResponse)
1299 | async def get_audit_logs(entity_type: str | None = Query(None), action: str |
None = Query(None),
1300 |                          username: str | None = Query(None), limit: int =
Query(50, ge=1, le=200),

```

// ===== Page 26 =====

```

1301 |         offset: int = Query(0, ge=0), db: AsyncSession =
Depends(get_db), _user=Depends(get_current_user)):
1302 |     items, total = await query_audit_logs(db, entity_type, action, username,
limit, offset)
1303 |     return {"items": items, "total": total, "limit": limit, "offset": offset}

```

// ===== 文件: backend/app/modules/admin/schemas.py =====

```

1304 | """AOI-Vision v0.1 - 管理模块 schemas
1305 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
1306 | """
1307 | from pydantic import BaseModel
1308 | from datetime import datetime
1309 |
1310 | class FieldOptionCreate(BaseModel):
1311 |     field_name: str
1312 |     option_value: str
1313 |     option_label: str
1314 |     sort_order: int = 0
1315 |
1316 | class FieldOptionUpdate(BaseModel):
1317 |     option_value: str | None = None
1318 |     option_label: str | None = None
1319 |     sort_order: int | None = None
1320 |     is_active: bool | None = None
1321 |
1322 | class FieldOptionResponse(BaseModel):
1323 |     id: int
1324 |     field_name: str
1325 |     option_value: str
1326 |     option_label: str
1327 |     sort_order: int
1328 |     is_active: bool
1329 |     model_config = {"from_attributes": True}
1330 |
1331 | class AuditLogResponse(BaseModel):
1332 |     id: str
1333 |     timestamp: datetime
1334 |     username: str
1335 |     action: str
1336 |     entity_type: str
1337 |     entity_label: str | None = None
1338 |     summary: str | None = None
1339 |     model_config = {"from_attributes": True}
1340 |
1341 | class AuditLogListResponse(BaseModel):
1342 |     items: list[AuditLogResponse]
1343 |     total: int
1344 |     limit: int
1345 |     offset: int

```



```
// ===== 文件: backend/app/modules/admin/service.py =====
```

```
1346 | """AOI-Vision v0.1 - 管理服务
1347 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
1348 | """
1349 | from sqlalchemy import select, desc, func
1350 | from sqlalchemy.ext.asyncio import AsyncSession
```

```
// ===== Page 27 =====
```

```
1351 | from app.modules.admin.models import FieldOption
1352 | from app.modules.admin.audit_model import AuditLog
1353 | from app.modules.admin.schemas import FieldOptionCreate, FieldOptionUpdate
1354 |
```

```
1355 | async def list_options(db: AsyncSession, field_name: str | None = None) ->
list[FieldOption]:
```

```
1356 |     stmt = select(FieldOption).where(FieldOption.is_active ==
True).order_by(FieldOption.sort_order)
1357 |     if field_name:
1358 |         stmt = stmt.where(FieldOption.field_name == field_name)
1359 |     return list((await db.execute(stmt)).scalars().all())
1360 |
```

```
1361 | async def create_option(db: AsyncSession, data: FieldOptionCreate) ->
FieldOption:
```

```
1362 |     opt = FieldOption(**data.model_dump())
1363 |     db.add(opt)
1364 |     await db.flush()
1365 |     return opt
1366 |
```

```
1367 | async def update_option(db: AsyncSession, opt_id: int, data: FieldOptionUpdate)
-> FieldOption | None:
```

```
1368 |     result = await db.execute(select(FieldOption).where(FieldOption.id ==
opt_id))
```

```
1369 |     opt = result.scalar_one_or_none()
1370 |     if not opt:
1371 |         return None
1372 |     for key, val in data.model_dump(exclude_none=True).items():
1373 |         setattr(opt, key, val)
1374 |     await db.flush()
1375 |     return opt
1376 |
```

```
1377 | async def delete_option(db: AsyncSession, opt_id: int) -> bool:
```

```
1378 |     result = await db.execute(select(FieldOption).where(FieldOption.id ==
opt_id))
```

```
1379 |     opt = result.scalar_one_or_none()
1380 |     if not opt:
1381 |         return False
1382 |     opt.is_active = False
1383 |     await db.flush()
1384 |     return True
1385 |
```

```
1386 | async def seed_default_options(db: AsyncSession):
```

```
1387 |     """初始化默认枚举值"""
```

```
1388 |     defaults = [
1389 |         ("defect_type", "scratch", "划痕", 1),
1390 |         ("defect_type", "dent", "凹陷", 2),
1391 |         ("defect_type", "stain", "污渍", 3),
1392 |         ("defect_type", "crack", "裂纹", 4),
1393 |         ("defect_type", "burr", "毛刺", 5),
1394 |         ("defect_severity", "critical", "严重", 1),
1395 |         ("defect_severity", "major", "主要", 2),
1396 |         ("defect_severity", "minor", "次要", 3),
1397 |         ("product_model", "model-a", "产品A型", 1),
1398 |         ("product_model", "model-b", "产品B型", 2),
1399 |         ("production_line", "line-1", "产线1号", 1),
1400 |         ("production_line", "line-2", "产线2号", 2),
```

```
// ===== Page 28 =====
```

```

1401 |     ]
1402 |     for fn, ov, ol, so in defaults:
1403 |         existing = await
db.execute(select(FieldOption).where(FieldOption.field_name == fn,
FieldOption.option_value == ov))
1404 |         if not existing.scalar_one_or_none():
1405 |             db.add(FieldOption(field_name=fn, option_value=ov, option_label=ol,
sort_order=so))
1406 |         await db.flush()
1407 |
1408 | async def query_audit_logs(db: AsyncSession, entity_type: str | None = None,
action: str | None = None,
1409 |                             username: str | None = None, limit: int = 50,
offset: int = 0) -> tuple[list, int]:
1410 |     stmt = select(AuditLog)
1411 |     count_stmt = select(func.count()).select_from(AuditLog)
1412 |     if entity_type:
1413 |         stmt = stmt.where(AuditLog.entity_type == entity_type)
1414 |         count_stmt = count_stmt.where(AuditLog.entity_type == entity_type)
1415 |     if action:
1416 |         stmt = stmt.where(AuditLog.action == action)
1417 |         count_stmt = count_stmt.where(AuditLog.action == action)
1418 |     if username:
1419 |         stmt = stmt.where(AuditLog.username == username)
1420 |         count_stmt = count_stmt.where(AuditLog.username == username)
1421 |     stmt = stmt.order_by(desc(AuditLog.timestamp)).offset(offset).limit(limit)
1422 |     items = list((await db.execute(stmt)).scalars().all())
1423 |     total = (await db.execute(count_stmt)).scalar() or 0
1424 |     return items, total

```

// ===== 文件: backend/app/modules/auth/__init__.py =====

```

1425 | """AOI-Vision v0.1
1426 | """
1427 | from app.modules.auth.router import router
1428 |
1429 | __all__ = ["router"]

```

// ===== 文件: backend/app/modules/auth/models.py =====

```

1430 | """AOI-Vision v0.2 - RBAC 权限模型
1431 | 日期: 2026-06-17 | 作者: William Chao / OASYS CORE
1432 | 描述: 全多对多 RBAC - 权限组 ↔ 账号组 ↔ 用户
1433 |         Permission Group (权限组) ←M2M→ Account Group (账号组) ←M2M→ User (用户)
1434 | """
1435 | import uuid
1436 | from sqlalchemy import Boolean, ForeignKey, String, Table, Column, JSON
1437 | from sqlalchemy.orm import Mapped, mapped_column, relationship
1438 | from app.core.database import Base, GUID
1439 | from app.shared.models import TimestampMixin
1440 |
1441 | # ===== M2M 关联表 =====
1442 |
1443 | user_account_groups = Table(
1444 |     "user_account_groups", Base.metadata,
1445 |     Column("user_id", GUID, ForeignKey("users.id", ondelete="CASCADE"),
primary_key=True),
1446 |     Column("account_group_id", GUID, ForeignKey("account_groups.id",
ondelete="CASCADE"), primary_key=True),
1447 | )
1448 |
1449 | account_group_permissions = Table(
1450 |     "account_group_permissions", Base.metadata,

```

// ===== Page 29 =====

```

1451 |     Column("account_group_id", GUID, ForeignKey("account_groups.id",
ondelete="CASCADE"), primary_key=True),

```

```

1452 |         Column("permission_id", GUID, ForeignKey("permissions.id",
ondelete="CASCADE"), primary_key=True),
1453 |     )
1454 |
1455 |
1456 | # ===== 权限组 =====
1457 |
1458 | class Permission(Base):
1459 |     """权限组 - 定义一组操作权限"""
1460 |     __tablename__ = "permissions"
1461 |
1462 |     id: Mapped[uuid.UUID] = mapped_column(GUID, primary_key=True,
default=uuid.uuid4)
1463 |     name: Mapped[str] = mapped_column(String(100), unique=True, nullable=False)
1464 |     description: Mapped[str] = mapped_column(String(500), default="")
1465 |     # permissions dict:
{"inspection": "manage", "measurement": "read", "defects": "manage", ...}
1466 |     permissions: Mapped[dict] = mapped_column(JSON, default=dict)
1467 |     is_active: Mapped[bool] = mapped_column(Boolean, default=True)
1468 |
1469 |     account_groups: Mapped[list["AccountGroup"]] = relationship(
1470 |         "AccountGroup", secondary=account_group_permissions,
back_populates="permissions"
1471 |     )
1472 |
1473 |
1474 | # ===== 账号组 =====
1475 |
1476 | class AccountGroup(Base):
1477 |     """账号组 - 用户逻辑分组 (如: 质检员组、管理员组、操作员组) """
1478 |     __tablename__ = "account_groups"
1479 |
1480 |     id: Mapped[uuid.UUID] = mapped_column(GUID, primary_key=True,
default=uuid.uuid4)
1481 |     name: Mapped[str] = mapped_column(String(100), unique=True, nullable=False)
1482 |     description: Mapped[str] = mapped_column(String(500), default="")
1483 |     is_active: Mapped[bool] = mapped_column(Boolean, default=True)
1484 |
1485 |     users: Mapped[list["User"]] = relationship(
1486 |         "User", secondary=user_account_groups, back_populates="account_groups"
1487 |     )
1488 |     permissions: Mapped[list["Permission"]] = relationship(
1489 |         "Permission", secondary=account_group_permissions,
back_populates="account_groups"
1490 |     )
1491 |
1492 |
1493 | # ===== 用户 =====
1494 |
1495 | class User(TimestampMixin, Base):
1496 |     __tablename__ = "users"
1497 |
1498 |     id: Mapped[uuid.UUID] = mapped_column(GUID, primary_key=True,
default=uuid.uuid4)
1499 |     username: Mapped[str] = mapped_column(String(50), unique=True,
nullable=False)
1500 |     hashed_password: Mapped[str] = mapped_column(String(255), nullable=False)

// ===== Page 30 =====

```